

Módulo 6

M6.1 Herança e Polimorfismo

Herança

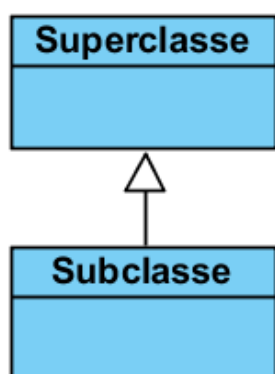
A Herança é um princípio da Programação Orientada a Objetos que permite que as classes compartilhem atributos e operações baseados em um relacionamento, geralmente generalização. A herança permite a criação de subclasses que herdam atributos e operações (ou Métodos) da classe pai (super-classe ou classe base). A herança é um conceito aplicado no momento de criação das classes. Ela é usada na intenção de evitar que classes que possuam atributos ou métodos semelhantes sejam repetidamente criados.

A herança permite que você baseie a definição de uma nova classe em uma classe previamente existente. Quando você baseia uma classe em outra, a definição da nova classe herda automaticamente todos os atributos, comportamentos e implementações presentes na classe previamente existente (SINTES, 2002, p. 72).

Nem sempre o mecanismo de delegação é o mais natural para reutilização de classes já existentes, embora seja simples. Em especial, quando queremos usar uma classe para servir de base à criação de outra mais especializada, a relação de composição imposta pelo uso do mecanismo de delegação acaba por criar soluções pouco naturais. Este recurso é utilizado sempre quando há necessidade de definir uma nova classe que seja um caso particular da classe original.

A solução é a reutilização classes através do mecanismo de herança, em que podemos criar uma classe usando outra como base e descrevendo ou implementando as diferenças e adições da classe usada como base, reutilizando os campos e métodos não privados da classe base. O mecanismo de herança é o mais apropriado para criar relações “é-um” entre classes.

No jargão das linguagens OO a classe base, aquela que serve para a criação de classes mais especializadas é chamado de Superclasse (ou Classe Pai, ou Classe Base), e a classe que herda as características da superclasse é chamado de Subclasse. Em UML a relação de herança é representada por uma seta



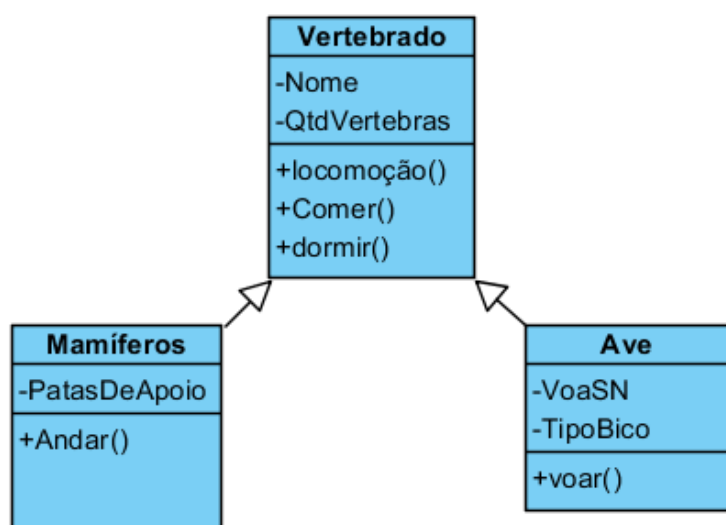
Herança não é cópia, é uma derivação! Alterações realizadas na classe pai poderá afetar a classe filha, porém a classe filha poderá ser definida com comportamentos distintos da classe pai.

Assim podemos dizer que as principais características da Herança são:

A reutilização do código de uma determinada classe em outras classes

- É um conceito chave usado na programação orientada a objetos para descrever uma relação entre as classes;
- Por meio da herança uma classe copia ou herda todas as propriedades, atributos e métodos de uma outra classe, podendo assim estender sua funcionalidade;
- A classe que cede os membros para a outra classe é chamada superclasse, classe pai ou classe base;
- A classe que herda os membros da outra classe é chamada subclasse ou classe derivada;
- A herança permite a reutilização de código e especifica um relacionamento de especialização/generalização do tipo "é um";

A Herança permite que você baseie a definição de uma nova classe em uma previamente existente. Assim podemos definir uma **Superclasse** chamado Vertebrados que reúne os atributos gerais que, no exemplo, os as sub classes mamíferos e aves possuem, e cada uma das subclasses possuem as diferenças entre eles:



Exemplo de Herança, a superclasse dos vertebrados e duas das suas subclasses

Em C# a identificação de herança acontece colocando o nome da superclasse logo após a declaração da classe que será a subclasse

```

class Filho : Pai

```

Nome da superclasse

Para entender o funcionamento do mecanismo de herança vamos passo a passo montar o processo. Inicialmente vamos construir uma classe simples chamado filho com um construtor que mostrará o texto "construtor Filho", e um método chamado junior, que mostrará na tela "sou o método Junior"

Analisando o funcionamento

```

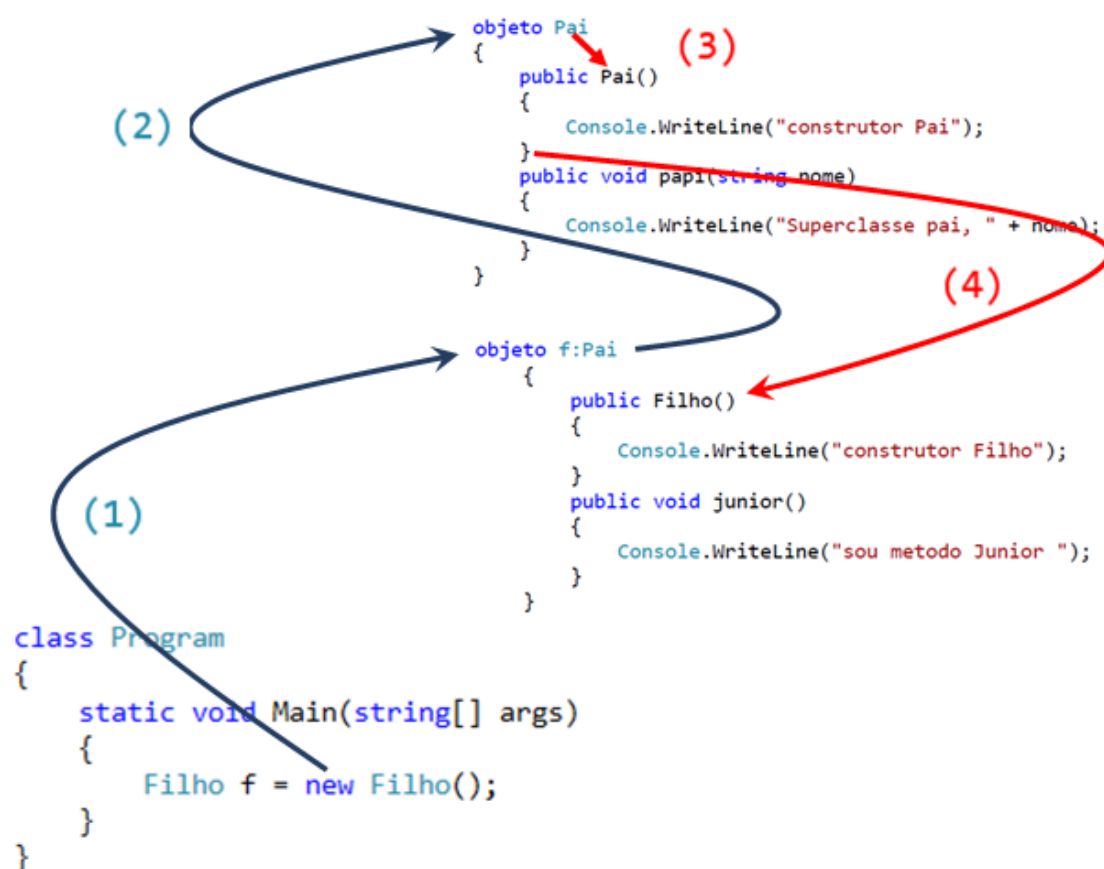
class Pai
{
    public Pai()
    {
        Console.WriteLine("construtor Pai");
    }
    public void papi(string nome)
    {
        Console.WriteLine("Superclasse pai, " + nome);
    }
}

```

```

class Filho:Pai
{
    public Filho()
    {
        Console.WriteLine("construtor Filho");
    }
    public void junior()
    {
        Console.WriteLine("sou metodo Junior ");
    }
}

```



A Classe Filho ao ser instanciada (1), pelo fato de ter a declaração de herança ":Pai", instancia a classe Pai (2). Assim como o todo processo de instanciação, a classe Pai executa o construtor (3). Uma vez terminado o construtor, o comando é devolvido para a classe Filho, que só então executa o seu construtor (4).

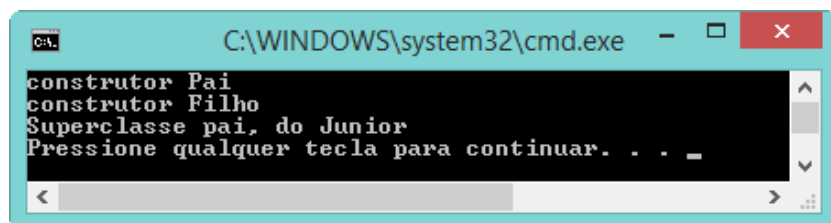
Assim se modificarmos o main executando a instância do filho utilizando um método do pai:

```

class Program
{
    static void Main(string[] args)
    {
        Filho f = new Filho();
        f.papi("do Junior");
    }
}

```

desta forma temos a saída



Como exemplo pode-se observar as classes 'aluno' e 'professor', onde ambas possuem atributos como nome, endereço e telefone. Nesse caso pode-se criar uma nova classe chamada por exemplo, 'pessoa', que contenha as semelhanças entre as duas classes, fazendo com que aluno e professor herdem as características de pessoa, desta maneira pode-se dizer que aluno e professor são subclasses de pessoa. A implementação ficaria da seguinte forma:

Definimos a classe Pessoa e seus respectivos atributos

```

//Classe pessoa
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ExemploHeranca
{
    public class Pessoa
    {
        private string _nome;
        public string Nome
        {
            get { return _nome; }
            set { _nome = value; }
        }

        private string _endereco;
        public string Endereco
        {
            get { return _endereco; }
            set { _endereco = value; }
        }

        private string _telefone;
        public string Telefone
        {
            get { return _telefone; }
            set { _telefone = value; }
        }
    }
}

```

Em seguida, definimos a classe Aluno, que herda de pessoa.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ExemploHeranca
{
    class Aluno:Pessoa
    {
    }
}
```

Por fim, definimos a classe Professor.

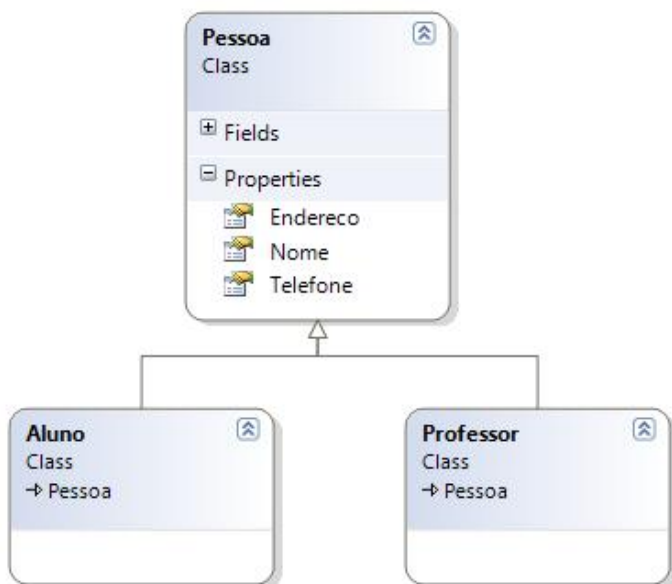
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ExemploHeranca
{
    public class Professor:Pessoa
    {
    }
}
```

Em termos de implementação utilizando a linguagem .Net C#, a instrução

- class Aluno:Pessoa é a sintaxe utilizada para representar a herança. Onde :
- class Aluno indica que o nome da classe é Aluno;
- :Pessoa indica que a classe mãe (de quem a classe filha herda) é a classe Pessoa.

Além disso, podemos observar esta mesma estrutura pelo diagrama de classe correspondente a implementação que acabamos de ver, conforme imagem abaixo:



• Referência à superclasse

A classes derivadas ou subclasses podem ter acesso a métodos das superclasses usando a palavra-chave *base*. O acesso a métodos de classes ancestrais é útil para aumentar a reutilização de código. Se existem métodos na classe ancestral que podem efetuar parte do processamento necessário, devemos usar o código que já existe em vez de reescrevê-lo

Algumas regras para o uso da palavra-chave *base* para chamar métodos de classes ancestrais como sub-rotinas são:

- Construtores são chamados simplesmente pela palavra-chave *base* seguida dos argumentos a serem passados para o construtor entre parênteses na declaração do construtor da subclasse, separado por *dois pontos* (:). Se não houver argumentos, a chamada deve ser feita como *:base()*.
- Métodos são chamados pela palavra-chave *base* seguida de um ponto e do nome do método. Se houver argumentos a serem passados para o método, estas devem estar entre parênteses, após o nome do método; caso contrário, os parênteses devem estar vazios.
- Construtores de superclasses só podem ser chamados de dentro de construtores de subclasses e, mesmo assim, somente se forem declarados na primeira linha de código do construtor da subclasse.
- Somente os métodos e construtores da superclasse imediata podem ser chamados usando a palavra-chave *base* – não existem construções como *base.base* que permitam a execução de métodos e construtores de classes ancestrais da classe ancestral. Caso seja necessário executar o construtor de uma classe ancestral da própria classe ancestral, os construtores podem ser escritos em cascata, de modo que o construtor da classe C chame o construtor da classe B que por sua vez chame o construtor da classe A.
- Se um método de uma classe ancestral for herdado pela classe descendente, ele pode ser chamado diretamente sem a necessidade da palavra-chave *base*.

Vamos tomar como exemplo a Classe Pai, e colocarmos uma sobrecarga de construtor.

```
class Pai
{
    public Pai()
    {
        Console.WriteLine("construtor Pai");
    }
    public Pai(string nome)
    {
        Console.WriteLine("Sou o construtor com parâmetros da classe Pai " + nome);
    }
    public void papi(string nome)
    {
        Console.WriteLine("Superclasse pai, " + nome);
    }
}
```

Na classe Filho colocamos também uma sobrecarga de construtor, porém este construtor irá direcionar para a classe Pai passando como parâmetro o valor recebido no processo de criação da instância pela chamada da *base*

```

class Filho : Pai
{
    public Filho()
    {
        Console.WriteLine("construtor Filho");
    }
    public Filho(string nome)
        : base(nome)
    {
    }
    public void junior()
    {
        Console.WriteLine("sou metodo Junior ");
    }
}

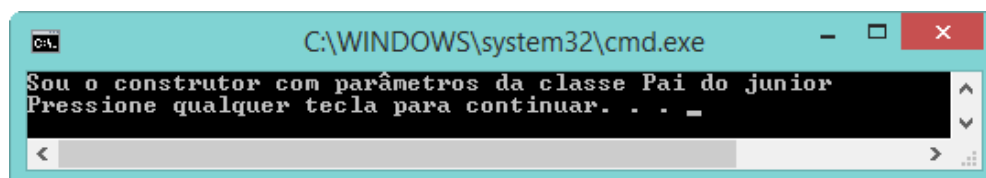
```

Assim ao criamos uma nova instância, o programa procura o construtor que possui a assinatura *string*, e este construtor invoca a superclasse através da chamada à base, procurando também o construtor pai que possui a assinatura correspondente. Desta forma é apresentada a saída que mostra o texto indicado pelo construtor cuja assinatura é uma *string* da superclasse

```

class Program
{
    static void Main(string[] args)
    {
        Filho f = new Filho("do junior");
    }
}

```



• Sobreposição e ocultação

Os métodos das classes específicas têm prioridade sobre os métodos das classes genéricas. Se o método chamado existir na classe filha ele é que será chamado, se não existir o método será procurado na classe mãe.

Quando definimos um método com a mesma assinatura na classe base e em alguma classe derivada, estamos aplicando o conceito de Reescrita de Método, Sobreposição ou Superposição.

A razão de sobrepormos métodos é que métodos de classes herdeiras geralmente executam tarefas adicionais que os mesmos métodos das classes ancestrais não executam.

A declaração de campos em uma classe descendente com o mesmo nome de campos declarados na classe ancestral chama-se ocultação. Ao contrário da sobreposição de métodos, que é bastante útil em classes herdeiras, a ocultação de campos não oferece muitas vantagens e as poucas oferecidas podem facilmente ser implementadas através de métodos que retornam valores e são superpostos de acordo com a necessidade.

As principais regras de sobreposição de métodos e ocultação de campos e regra de uso de modificadores de acesso em classes herdadas são:

- A sobreposição de um método em uma subclasse não elimina o acesso ao método de mesma assinatura na classe ancestral – este pode ser acessado, de dentro da classe herdeira, com a palavra chave *base*, contanto que não tenha sido declarado como *private*.
- Métodos declarados em uma subclasse com o mesmo nome mas assinaturas diferentes (por exemplo, número de argumentos diferentes) dos métodos da superclasse não sobrepõem estes métodos
- Métodos podem ser sobrepostos com diferentes modificadores de acesso, contanto que os métodos sobrepostos tenham modificadores de acesso menos restritivos. Em outras palavras, podemos declarar um método na superclasse com o modificador *private* e sobrepor este método em uma subclasse com o modificador *public*, mas não podemos fazer o contrário.
- Métodos estáticos declarados em classes ancestrais não podem ser sobrepostos em classes descendentes, nem mesmo se não forem declarados como estáticos.
- Se um campo é declarado em uma superclasse e oculto em subclasses, e métodos que acessam este campo são herdados, estes métodos farão referência ao campo da classe onde foram declarados.

Por padrão, as implementações dos métodos de uma superclasse não podem ser substituídas pelas subclasses. Para alterar esse padrão, devemos acrescentar o modificador *virtual*. e *override* na que tem a mesma assinatura.

Para aplicar o conceito de sobreposição, vamos montar um programa com duas classes. A primeira classe armazena o nome e o salário de um empregado, e retorna o valor do seu salário. A segunda classe é de um comissionado, ou seja, uma especialização da classe dos empregados, já que além de receber um salário ele recebe uma comissão extra. Por esta configuração, não é necessário criar uma classe com todas as informações de um empregado, bastando então utilizar o a classe Empregado como superclasse da classe Comissionado.

```
class Empregado
{
    public string nome;
    private double SalarioBase;
    public Empregado(string nome, double SB)
    {
        this.nome = nome;
        this.SalarioBase = SB;
    }
    public virtual double CalculaPagto()
    {
        return SalarioBase;
    }
}
```



```

class Comissionado : Empregado
{
    private double comissao;
    public Comissionado(string nome, double SB,
        double comissao)
        : base(nome, SB)
    {
        this.comissao = comissao;
    }
    public override double CalculaPagto()
    {
        return base.CalculaPagto()+comissao;
    }
}

```

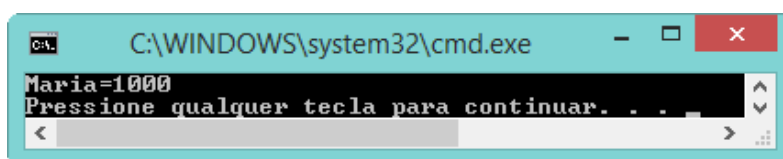
Ambas as classes possuem o método `CalculaPagto()`, se considerarmos sem as palavras chaves *virtual* e *override*, para uma classe externa haveria um sério problema. Como a classe `Comissionado` tem o método `CalculaPagto()` e ele estende a classe `Empregado` que também tem o mesmo método com a mesma assinatura a visão externa seria de uma classe com dois métodos iguais. Desta forma para uma referência externa, utilizam-se as palavras *virtual* para o método sobreposto e *override* para o método superposto. A linguagem C# não obriga o uso das palavras sem resultar em erro, porém o editor avisa (warning) que existe um problema.

Ao executarmos o programa criando a instância para um empregado normal, o método `CalculaPagto()` é o da classe `Empregado`, resultando apenas no salário base

```

class Program
{
    static void Main(string[] args)
    {
        Empregado e1 = new Empregado("Maria", 1000);
        Console.WriteLine("{0}={1}", e1.nome, e1.CalculaPagto());
    }
}

```

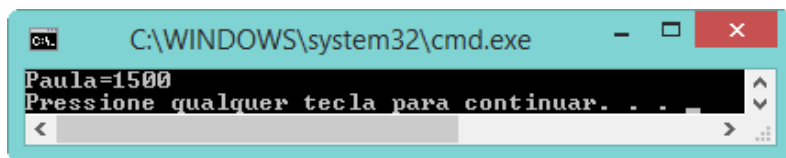


Alterando a classe `Program` para criar a instância de um `comissionado` o `CalculaPagto()` executado é o da classe `Comissionado`, e não da classe `Empregado` (que é utilizado apenas para calcular o salário base). Desta forma na saída do programa o salário calculado é a soma do salário base com a comissão .

```

class Program
{
    static void Main(string[] args)
    {
        Comissionado c1 = new Comissionado("Paula", 1000, 500);
        Console.WriteLine("{0}={1}", c1.nome, c1.CalculaPagto());
    }
}

```



Polimorfismo

Na programação orientada a objetos, o polimorfismo permite que referências de tipos de classes mais abstratas representem o comportamento das classes concretas que referenciam. Assim, um mesmo método pode apresentar várias formas, de acordo com seu contexto. O polimorfismo é importante pois permite que a semântica de uma interface seja efetivamente separada da implementação que a representa. O termo polimorfismo é originário do grego e significa "muitas formas" (poli = muitas, morphos = formas).

O polimorfismo está relacionado com o conceito de herança, especificamente em relação a métodos. O mecanismo de herança permite a criação de classes a partir de outras já existentes com relações "é-um-tipo-de", de forma que, a partir de uma classe genérica, classes mais especializadas possam ser criadas.

Quando uma nova classe necessita de todos os atributos e métodos de uma classe já existente, porém a nova classe possui a execução de um ou mais métodos diferenciados é possível herdarmos todos os métodos e atributos da classe original e realizar a alteração do comportamento do método somente na nova classe.

A partir desse momento, todo objeto que se utilizar da nova classe e executar o método em questão, será executado o método novo e não o método original. O polimorfismo permite a manipulação de instâncias de classes que herdam de uma mesma classe ancestral de forma unificada: podemos escrever métodos que recebam instâncias de uma classe C, e os mesmos métodos serão capazes de processar instâncias de qualquer classe que herde da classe C, já que qualquer classe que herde de C é-um-tipo-de C.

Polimorfismo, ter muitas formas. Em termos de programação, muitas formas significa que um único nome pode representar um código diferente, selecionado por algum mecanismo automático. Assim, o polimorfismo permite que um único nome expresse muitos comportamentos diferentes. (Sintes, 2002, p. 122)

Para exemplificar, observe o código abaixo :
Suponha a seguinte classe escrita em .Net C#:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ExemploPolimorfismo
{
    public abstract class OperacaoMatematica
    {
        public abstract double calcular(double x, double y);
    }
}
```

Esta é uma classe abstrata que representa qualquer operação matemática. Podemos imaginar diversas operações que se encaixam na sua interface, como soma, subtração, multiplicação ou divisão, entre outras. Note que, mesmo que a natureza do cálculo mude, a semântica do método calcular não muda, ou seja, ele sempre calculará o resultado da

operação matemática que está sendo trabalhada.

Definamos então, duas subclasses, Soma e Subtracao, que implementam a classe OperacaoMatematica:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ExemploPolimorfismo
{
    public class Soma:OperacaoMatematica
    {
        public override double calcular(double x, double y)
        {
            return x + y;
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ExemploPolimorfismo
{
    public class Subtracao : OperacaoMatematica
    {
        public override double calcular(double x, double y)
        {
            return x - y;
        }
    }
}
```

O seguinte trecho de código demonstra o uso do polimorfismo:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ExemploPolimorfismo
{
    public class Contas
    {
        public static void mostrarCalculo(OperacaoMatematica operacao, double x, double y)
        {
            Console.WriteLine("Resultado:" + operacao.calcular(x, y));
        }
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
```

```
namespace ExemploPolimorfismo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Primeiro calculamos uma soma
            Contas.mostrarCalculo(new Soma(), 5, 5); //Imprime o resultado é: 10
            //Depois uma subtração
            Contas.mostrarCalculo(new Subtracao(), 5, 5); //Imprime o resultado é: 0

            Console.ReadLine();
        }
    }
}
```

Note que, embora o método calcular tenha sido chamado duas vezes no interior de `mostrarCalculo`, o comportamento apresentado variou de acordo com a classe ao qual ele representava no momento. É comum definir sobrecarga de métodos ou simplesmente sobrecarga como uma forma de polimorfismo embora esta definição deixe lacunas conceituais.

Vantagens na utilização do Polimorfismo

Em linguagens de programação não polimórficas, para implementar o método `mostrarCalculo`, seria necessário recorrer a uma enumeração com o tipo de operação e, dentro do método, testar o valor da enumeração com um `case`, como no exemplo abaixo:

```
public void mostrarCalculo (int operacao, double x, double y)
{
    System.out.print("O resultado é: ");
    switch (operacao)
    {
        case SOMA:
            System.out.print(""+(x+y));
            break;
        case SUBTRACAO:
            System.out.print(""+(x-y));
            break;
        //... outras operacoes
        default: throw new UnsupportedOperationException()
    }
}
```

Além do código ser maior e mais difícil de ler, essa implementação tem outros problemas. Provavelmente esse não será o único método a utilizar operações matemáticas e, portanto, pode-se esperar não um, mas vários `switchs` como esse pelo código. O que acontece, então, se uma nova operação for adicionada ao sistema? Será necessário que todos os `switchs` sejam encontrados e substituídos. Com o polimorfismo, a modificação restringiria-se apenas a criação de uma nova classe.

M6.2 Persistência

Uma vez definido as classes e essas classes são utilizados dentro do programa em forma de objetos, tais objetos podem assumir duas formas de existência dentro do programa: objetos

transientes ou objetos persistentes.

Objetos transientes são dados que trabalham em meios voláteis, ou seja, em memória. Uma vez finalizada a execução do programa, todos os dados referentes ao objeto serão perdidos, não podendo ser recuperados.

Objetos persistentes são dados que trabalham em meios não-voláteis, ou seja, seus dados são armazenados em disco através de arquivos ou banco de dados. Isso permite que uma vez finalizada a execução do programa os dados do objeto pode ser recuperado na próxima execução do programa.

Está técnica também pode ser aplicada para transferência de dados, onde se deseja transferir os dados de um objeto para outros programas de maneira fidedigna. Tal transferência é útil para a comunicação entre aplicativos e para a replicação de dados, especialmente em casos de replicação de dados assíncrona.

Desta forma, quando iniciamos o aprendizado de POO, é muito comum não utilizarmos persistência, deixando esta técnica para estudos mais avançados.

Em orientação a objetos, o termo persistência refere-se ao intervalo de tempo entre a criação e destruição de um objeto. Um objeto persistente não possui seu ciclo de vida atrelado à execução do aplicativo que o gerou. Para isso, deve habitar um meio de armazenamento permanente, ou seja, a memória secundária.

Objetos devem, ainda, possuir uma identidade: alguma coisa sobre si próprio, que seja única e invariável, que permitirá distinção entre objetos diferentes.

Linguagens como C#, Java, entre outras possuem recursos de limpeza automático de memória - Garbage Collector (Coletor de Lixo). O Garbage Collector (GC) tem como tarefa identificar objetos que foram instanciados e não estão mais em uso.

Além disso, o GC pode ser chamado manualmente, via aplicação. O uso do GC deve ser feito de forma cautelosa, tendo em vista que uma vez que o GC é chamado para excluir determinado objeto em memória, o mesmo não poderá ser recuperado ou reutilizado.

Exercício 1:

B2-1

Herança pode ser definida como :

A)

princípio da Programação Orientada a Aspectos que permite que as classes compartilhem atributos e operações baseados em um relacionamento, geralmente generalização

B)

princípio da Programação Orientada a Objetos que permite que as classes compartilhem métodos e operações baseados em um relacionamento, geralmente generalização

C)

princípio da Programação Orientada a Objetos que permite que as classes compartilhem atributos e propriedades baseados em um relacionamento, geralmente generalização

D)

princípio da Programação Orientada a Objetos que permite que os objetos compartilhem atributos e operações baseados em um relacionamento, geralmente composição

E)

princípio da Programação Orientada a Objetos que permite que as classes compartilhem atributos e operações baseados em um relacionamento, geralmente generalização

Exercício 2:

B2-1-3

Sobre o conceito de Herança é correto afirmar :

A)

Ela é usada na intenção de evitar que objetos que possuam atributos ou métodos semelhantes sejam repetidamente criados

B)

Ela é usada na intenção de evitar que classes que possuam atributos semelhantes sejam repetidamente criados

C)

Ela é usada na intenção de evitar que classes que possuam métodos semelhantes sejam repetidamente criados

D)

Ela é usada na intenção de evitar que classes que possuam atributos ou métodos semelhantes sejam repetidamente criados

E)

Ela é usada para que um atributo possa ser utilizada em todas as classes

Exercício 3:

B2-1 4

Qual dos trechos de código abaixo representam a implementação de Herança :

A)

class Subtracao : OperacaoMatematica

B)

void Subtracao : OperacaoMatematica

C)

class Subtracao - OperacaoMatematica

D)

class Subtracao

E)

class OperacaoMatematica extends Subtracao

Exercício 4:

Entre as principais vantagens do Polimorfismo, podemos destacar :

A)

Permite que a semântica de uma interface seja efetivamente separada da implementação que a representa

B)

Permite que a instância de uma classe seja efetivamente separada da implementação que a representa

C)

Permite que as propriedades um objeto seja efetivamente separada da implementação que a representa

D)

Permite que os métodos um objeto seja efetivamente separado da implementação que o representa

E)

Permite que os métodos tenham nomes diferentes

Exercício 5:

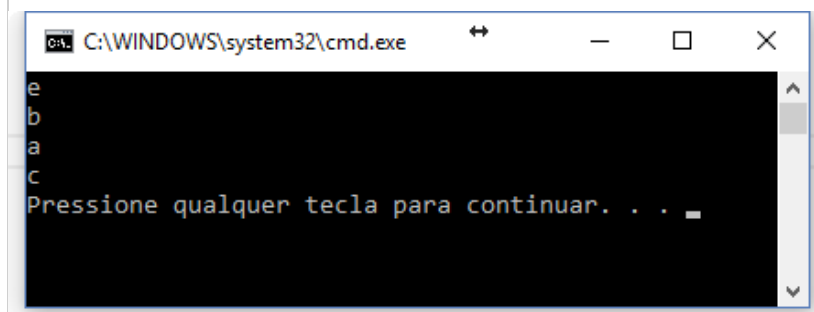
B2-1 8

Dado p código abaixo:

```
1      class Classe1
2      {
3          public Classe1()
4          {
5              Console.WriteLine("a");
6          }
7          public Classe1(String st)
8          {
9              Console.WriteLine(st+"\nb");
10         }
11     }
12     class Classe2 : Classe1
13     {
14         public Classe2()
15         {
16             Console.WriteLine("c");
17         }
18         public Classe2(String st):base(st)
19         {
20             Console.WriteLine(st + "\nd");
21         }
22     }
23     class Program
24     {
25         static void Main(string[] args)
26         {
27             Classe1 c1 = new Classe1("e");
28             Classe2 c2 = new Classe2();
29         }
30     }
```

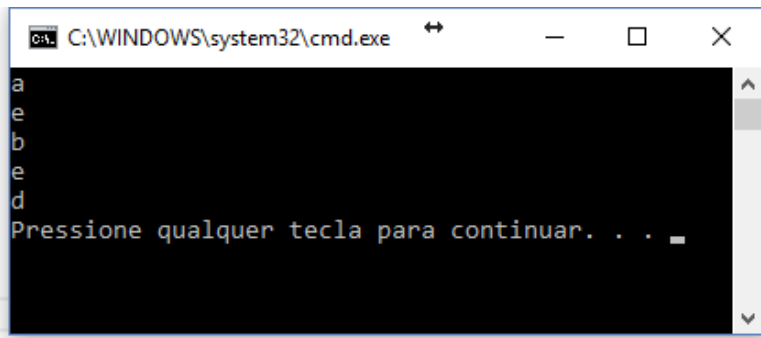
Qual a tela e a seqência de execução correta?

A)



27, c1-1, c1-7, c1-9, 28, c2-12, c2-1, c2-3, c2-5, c2-12, c2-14, c2-16

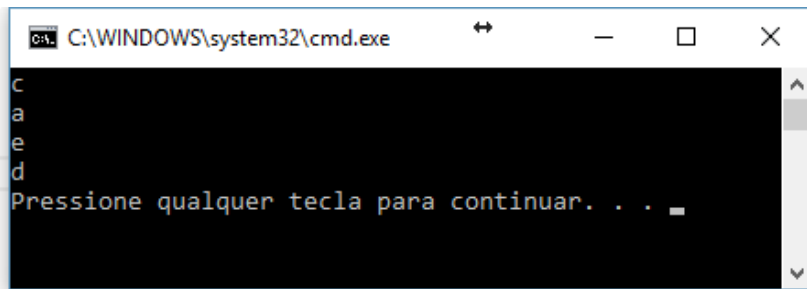
B)



```
C:\WINDOWS\system32\cmd.exe
a
e
b
e
d
Pressione qualquer tecla para continuar. . .
```

27-c1-1, c1-3, c1-5, 28, c2-12, c2-18, c2-1, c1-7, c2-9, c2-20

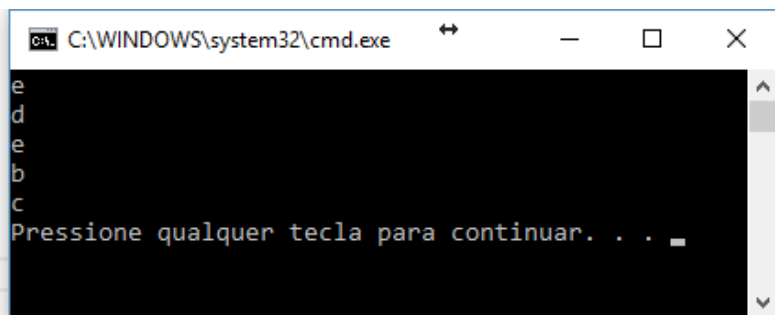
C)



```
C:\WINDOWS\system32\cmd.exe
c
a
e
d
Pressione qualquer tecla para continuar. . .
```

28, c1-1, c1-12, c1-14, c1-16, c1-1, c1-3, c1-5, 28, c2-12, c2-18, c2-20

D)



```
C:\WINDOWS\system32\cmd.exe
e
d
e
b
c
Pressione qualquer tecla para continuar. . .
```

28, c1-1, c1-7, c1-18, c1-20, c1-7-c1-9, 28, c1-12, c1-14, c1-16

E)

Erro, Não compila

Exercício 6:

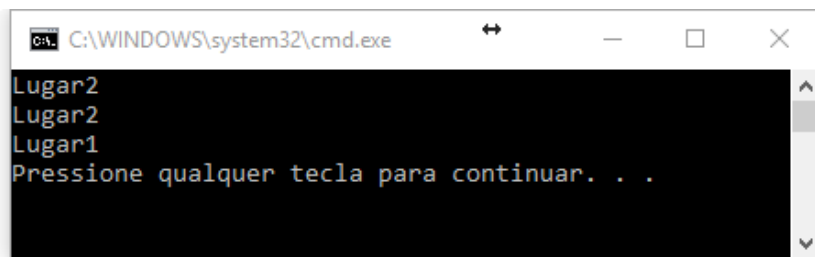
B2-1 9

Dado o código abaixo:

```
1      class Classe1
2      {
3          public Classe1()
4          {
5              Console.WriteLine("Lugar1");
6          }
7      }
8      class Classe2 : Classe1
9      {
10         public Classe2()
11         {
12             Console.WriteLine("Lugar2");
13         }
14     }
15     class Program
16     {
17         static void Main(string[] args)
18         {
19             Classe1 c1 = new Classe1();
20             Classe2 c2 = new Classe2();
21         }
22     }
```

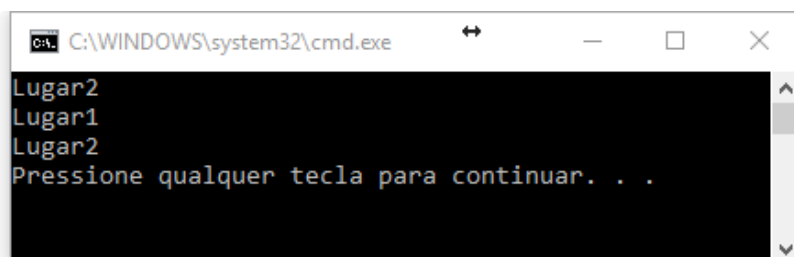
qual a saída correta?

A)



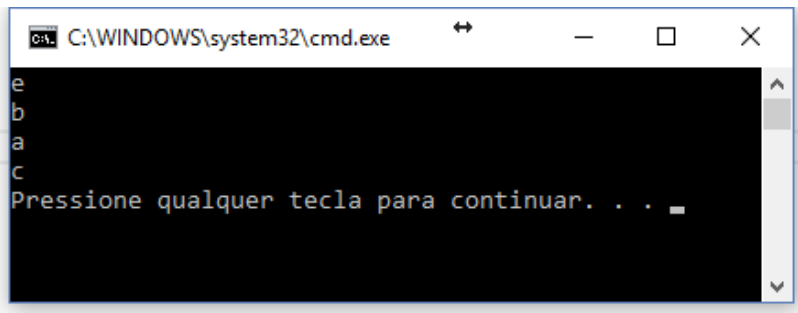
```
C:\WINDOWS\system32\cmd.exe
Lugar2
Lugar2
Lugar1
Pressione qualquer tecla para continuar. . .
```

B)



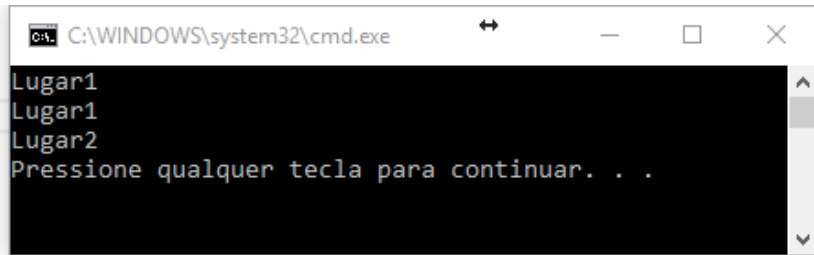
```
C:\WINDOWS\system32\cmd.exe
Lugar2
Lugar1
Lugar2
Pressione qualquer tecla para continuar. . .
```

C)



```
C:\WINDOWS\system32\cmd.exe
e
b
a
c
Pressione qualquer tecla para continuar. . . .
```

D)



```
C:\WINDOWS\system32\cmd.exe
Lugar1
Lugar1
Lugar2
Pressione qualquer tecla para continuar. . . .
```

E)

Erro, o programa não compila.

Exercício 7:

B2-2 1

Em orientação a objetos, o termo persistência refere-se a:

A)

intervalo de tempo entre a criação e destruição de um objeto. Um objeto persistente não possui seu ciclo de vida atrelado à execução do aplicativo que o gerou. Para isso, deve habitar um meio de armazenamento permanente, ou seja, a memória secundária.

B)

intervalo de tempo entre a criação e destruição de uma classe. Uma classe persistente não possui seu ciclo de vida atrelado à execução do aplicativo que o gerou. Para isso, deve habitar um meio de armazenamento permanente, ou seja, a memória secundária.

C)

intervalo de tempo entre a criação e destruição de um objeto. Um objeto persistente possui seu ciclo de vida atrelado à execução do aplicativo que o gerou. Para isso, deve habitar um meio de armazenamento permanente, ou seja, a memória secundária.

D)

intervalo de tempo entre a criação e destruição de um objeto. Um objeto persistente não possui seu ciclo de vida atrelado à execução do aplicativo que o gerou. Para isso, não deve habitar um meio de armazenamento intermediário, ou seja, a memória secundária.

E)

característica da POO que permite o programa executar métodos com o mesmo nome dentro de uma classe